

Senior ML Engineer - Technical Assessment

Jesta I.S. Inc.

Time Expected: 4-6 hours over 5 days

What to Submit: Code + Documentation + Short Presentation

What You're Evaluating

We need someone who can fix a real forecasting system we inherited from a vendor. This isn't a greenfield project—you'll be debugging production code, making pragmatic decisions under constraints, and communicating technical work to non-technical stakeholders.

We care about:

- How you approach messy, real-world problems
- Your judgment on what matters vs what doesn't
- Clean, maintainable code
- How you explain your thinking

Note: Some information is intentionally incomplete (like real projects). Ask questions if you need clarification—we'll respond within 24 hours.

Background

You're joining a team that forecasts demand for fashion retailers. Two clients: one sells athletic footwear/apparel (think Nike, Adidas), the other is luxury menswear. The current system has 50% error, and prospective clients are waiting for demos.

What's working:

- Daily sales data (3 years)
- Weekly retraining pipeline (Kedro + Polars)
- Models run, no crashes

What's broken:

- 50% WMAPE overall (should be 20-30%)
- Some categories worse: Footwear 233% error, Sportswear >1000% error
- System forecasts 12.6M products per run—most are dead stock or wrong assortments
- Memory issues (OOM crashes during batch processing)

Your constraints:

- 3 months to get demo-ready
- Junior developer on team (knows Python, learning ML)
- Must use Polars (not pandas—already in production)
- Cannot throw more money at AWS (cost-conscious)

Part 1: Strategy & Architecture (40%)

Task 1: Root Cause Memo

Time: ~45 minutes

Write a 1-page analysis answering:

- 1. What are the top 3 reasons for 50% error?**
Rank by impact. For each, explain: what's wrong, why it matters, rough impact estimate.
- 2. Why does Footwear have 233% error while Made-to-Measure has 40%?**
What would you investigate? What hypotheses do you have?
- 3. One critical decision you'd make in Week 1**
If you could only fix one thing first, what would it be and why?

Format: PDF or Markdown. Be concise—we read 20+ of these.

Task 2: 90-Day Plan

Time: ~60 minutes

Create an implementation roadmap with:

Month 1 Goal: [Specific WMAPE target]

- Week 1: [Task] → Expected impact: [metric]
- Week 2: [Task] → Expected impact: [metric]
- Week 3-4: [Tasks]
- Key risk: [what could go wrong] → Mitigation: [how you'd handle it]

Month 2 Goal: [Target]

- [Similar format]

Month 3 Goal: [Target]

- [Similar format]

Requirements:

- Each task needs: objective, expected impact, rough effort
- Show how you'd validate progress (what metrics you'd track)
- Include at least 2 risks with mitigation plans
- Be specific—"improve features" is too vague

Format: Document or spreadsheet. Visual timeline welcome but not required.

Task 3: System Design

Time: ~45 minutes

The current system forecasts products that shouldn't be forecasted (e.g., items sitting 8 months with no sales, ecom getting all 150k products even though only 15k are online).

Design a "Product Qualification" component that fixes this.

Include:

1. Inputs/Outputs

What data goes in? What comes out? How does it fit into existing pipeline?

2. Business Rules

How do you decide what to forecast? Different logic for:

- Physical stores vs ecom

- New products vs mature products
- High vs low volume categories

3. **Scale**

How do you handle 150k SKUs × 61 stores without OOM?

4. **Validation**

How do you know it's working? What would you measure?

Deliverable: 2-3 page design doc. Diagrams encouraged. Focus on your reasoning—why these choices?

Part 2: Hands-On Engineering (60%)

We've provided a CSV file with 100 days of data for one product at one store.

Columns: date, style, site, sales, inventory, original_price, current_price, cost, launch_date

What's in the data:

- Normal sales period (first 40 days)
- Stockout period (days 41-50, inventory=0)
- Markdown period (days 70+, price drops)
- Some data quality issues (intentional—reflect reality)

Task 4: Analysis & Features

Time: ~90 minutes

Part A: Exploration (20 min)

Load the data. Create 2-3 visualizations showing:

- What patterns do you see?
- What problems exist?
- What would affect forecast accuracy?

Part B: Feature Engineering (60 min)

Implement these features (choose 3 of 4):

1. **Stockout Detection**

Flag periods where inventory=0 but there were recent sales

2. **Product Lifecycle**

Calculate weeks since launch, classify as: new/growth/mature/clearance

3. **Sales Momentum**

Compare recent trend (7-day) vs longer trend (28-day)

4. **Inventory Health**

Calculate: days of supply, markdown depth, margin status

Part C: Product Qualification (20 min)

Write a function that returns True/False for "should we forecast this product?"

Consider: sales recency, inventory age, profitability, data quality.

Deliverable: Jupyter notebook or Python script with clear comments.

Task 5: Model Comparison

Time: ~60 minutes

Using the sample data:

1. **Split data properly**

Train on first 80 days, test on last 20 days

2. **Build 3 approaches:**

- Baseline: 28-day moving average
- LGBM: Using only lag features (7d, 14d, 28d)
- LGBM: Using your engineered features

3. **Compare results:**

- Calculate WMAPE for each
- Show where features helped most

- Brief analysis: what worked, what didn't, what next

Deliverable: Code + results table/chart + 1 paragraph analysis.

Task 6: Production Quality

Time: ~30 minutes

Take ONE function from Task 4 and make it production-ready:

- Type hints
- Docstring with example
- Input validation
- Error handling
- At least 2 unit tests

Example structure:

```
python

def detect_stockouts(
    df: pl.DataFrame,
    lookback_days: int = 7
) -> pl.DataFrame:
    """
    Detect periods where inventory=0 but recent sales>0.

    Args:
        df: Sales data with columns [date, sales, inventory]
        lookback_days: Days to look back for recent sales

    Returns:
        DataFrame with added column 'is_stockout'

    Example:
        >>> df = detect_stockouts(sales_df, lookback_days=7)
        >>> df.filter(pl.col('is_stockout'))
    """
    # Your implementation
    pass

def test_stockouts_basic():
    """Test clear stockout case"""
    pass

def test_stockouts_edge_cases():
    """Test no data, missing columns, etc"""
    pass
...

---
```

```
## What to Submit

**Package everything as:**
...

your_name_ml_assessment/
├── README.md           # How to run your code
├── strategy_analysis.pdf # Part 1: Tasks 1-3
├── engineering.ipynb    # Part 2: Tasks 4-5
├── production_code.py   # Part 2: Task 6
├── requirements.txt     # Dependencies
├── presentation.pdf     # 10-12 slides
└── data/               # Include the CSV we sent
```

Submit via: GitHub repo (preferred) or ZIP to [email]

Presentation Prep

If we move forward, you'll present your work in a 45-min call:

- 20 min: Walk us through your approach
- 15 min: Deep-dive Q&A
- 10 min: Your questions for us

What to cover:

- Problem as you understood it
- Your strategy and why
- Key technical decisions
- Results and next steps

Tips:

- Tell a story (problem → approach → results)
- Show, don't tell (charts > bullet points)
- Explain your reasoning
- Be ready to defend choices

We care more about how you think than perfection. Use the presentation to show your judgment and communication skills.

Evaluation

What we're looking for:

Strategy (40%):

- Did you identify the real problems or just symptoms?
- Is your roadmap realistic and well-prioritized?
- Does your design address scale and maintainability?

Engineering (60%):

- Clean, working code
- Thoughtful feature engineering
- Proper validation methodology
- Production-quality standards

Bonus points:

- Clear communication
- Good judgment calls
- Questions that show deep thinking
- Code we'd actually want to deploy

Instant disqualifiers:

- Code doesn't run
- No documentation
- Plagiarism (must be your own work, no AI)
- Missing major deliverables

FAQ

Q: Can I use pandas instead of Polars?

A: For the take-home, yes. But explain why and how you'd transition to Polars.

Q: What if I need more time?

A: Ask for an extension—life happens. We'd rather see good work than rushed work.

Q: Can I use ChatGPT/Claude?

A: Yes, but the code and thinking must be yours. We'll ask about your decisions.

Q: What libraries can I use?

A: Whatever you want. Just include in requirements.txt.

Q: The data looks weird in places—is that intentional?

A: Yes. Real data is messy. Handle it.

Q: How much code do you expect?

A: Quality over quantity. A well-documented 200-line solution beats 1000 lines of spaghetti.

Next Steps

Timeline:

1. You submit (within 5 days)
2. We review (2-3 days)
3. Presentation round if selected (45 min)
4. Decision (1-2 days after presentation)

What happens if not selected:

- We send detailed feedback (we respect the time you invested)
- Keep in touch—we're always hiring

Good luck. Show us how you think.

Sample Data

fashion_sample.csv (attached)

100 days of sales for one product at one store. Includes normal period, stockout, and markdown phases. Some data quality issues included intentionally.

This assessment reflects real problems we're solving. We know it's substantial work—that's why we provide detailed feedback to everyone who completes it, whether hired or not.